

Technical: Shristi Feed Data Documentation

Last updated by | shah saumil (IT Apps, KSL) | Jun 23, 2026 at 11:01 AM GMT+5:30

Market Data WebSocket Feed — Protocol Guide (`native_batch`)

This document describes how to connect to and consume the market-data WebSocket feed service. It is **language-agnostic**: you can implement a client in any language with a WebSocket library and the ability to read little-endian binary data.

Wire structures are given in **C style with `#pragma pack(1)`** — every struct is tightly packed (no padding) and every multi-byte integer is **little-endian**.

Scope. This guide covers the `native_batch` mode only. Other modes exist but are **not** documented here:

- `ws_delta_batch` — same wire protocol plus a bitmask-based incremental (delta) encoding.

If you need either of those, ask for the corresponding spec.

1. Overview

- A single **WebSocket** connection carries both planes:
- **Control plane** — JSON **text** frames (auth, subscribe, unsubscribe, snapshot).
- **Data plane** — **binary** frames (`ArrayBuffer`), little-endian, often **batched**

(multiple packets concatenated into one frame).

- Set the socket's binary type to `arraybuffer` (or your language's raw-bytes equivalent) before reading data frames.

- All prices arrive as **scaled integers**. You divide by a per-exchange **divider** (delivered in the auth response) to get the real value. See [§6](#).

2. Connecting

2.1 URL

Take the HTTP base URL of the service, switch the scheme from `http / https` to `ws / wss`, and append the feed path. The default path is `/mfeed`.

```
https://<host> + /mfeed -> wss://<host>/mfeed
http://<host>:<port> + /mfeed -> ws://<host>:<port>/mfeed
```

No custom HTTP headers or sub-protocols are required for the handshake — authentication is performed by the first application message (below).

2.2 Authenticate

Immediately after the socket opens, send a single JSON **text** frame:

```
{
  "user": "<username>",
  "auth": "<password>",
  "format": "native_batch",
  "source": "NEOWEB",
  "sdk_version": 1.1,
  "sdk_date": "<build date string>"
}
```

Field	Meaning
user / auth	Your credentials.
format	Must be "native_batch" for the protocol in this document.
source / platform	Client identification. Use values assigned to you.
sdk_version / sdk_date	SDK/build identifiers (integer / string).

2.3 Auth response — store the dividers

The server replies with a JSON **text** frame carrying `message_code == 1117` :

```

{
  "message_code": 1117,
  "id": "<instance name>",
  "format": "native_batch",
  "exchanges": {
    "nse_cm": { "divider": 100 },
    "nse_fo": { "divider": 100 },
    "cde_fo": { "divider": 10000000 }
  }
}

```

- **Persist** `exchanges[*].divider` . You need the divider for an exchange to decode every integer price you later receive for that exchange (see [§6](#)).

- **Map exchange strings to enum values.** This JSON keys each exchange by its `**name string**` (`"nse_cm"` , `"nse_fo"` , ...). The binary feed, however, identifies the exchange by its **numeric enum value** in the header (`exchange_id` , e.g. `1` for `nse_cm`) — it does **not** send the name string. So store the dividers keyed by the enum value, or keep a name ↔ value map, so you can look up the right divider from the `exchange_id` byte of an incoming packet. The full name ↔ value mapping is in [§5](#).

- If `format` comes back as `"native_fallback"` the server has downgraded you to a different body layout that is **out of scope** for this document — treat it as an error unless you have the fallback spec.

2.4 Connection lifecycle

- **Error** during connect → the connection failed; surface it and retry.
- **Close** → reconnect from scratch: re-open, re-authenticate, and `**re-send all your subscriptions**` (the server does not remember them across connections).
- **Heartbeat** — there is no application-level ping/pong in this mode; rely on the WebSocket layer's native ping/pong to keep the connection alive.

3. Control Messages (Client → Server)

All control messages are JSON **text** frames.

3.1 Subscribe / Unsubscribe / Snapshot an instrument

```
{ "event": "<event>", "inputtoken": "<exchange>|<token>", "auction": false }
```

- inputtoken = "<exchange_string>|<token>" , e.g. "nse_cm|11536" .

Exchange strings are listed in [§5](#).

- auction — set true only to target the auction variant of an instrument; otherwise

false .

- Snapshot** requests the current full state **once**. The reply uses the ****same binary message types**** as the live feed (see [§7](#)).

The exact event value depends on the data **level** (depth) or whether the instrument is an index. Pick from this table:

Intent	Level	subscribe event	unsubscribe event	snapshot event
Index	— (type = Index)	subscribeIndices	unsubscribeIndices	snapshotIndices
Full depth	16	subscribeFullDepth	unsubscribeFullDepth	NA
Depth	8	subscribeDepth	unsubscribeDepth	snapshotDepth
Touch line	4	subscribeScrips	unsubscribeScrips	snapshotScrips
Mini touch line	1 (default)	subscribeScripsLite	unsubscribeScripsLite	snapshotScripsLite

Examples

```
{ "event": "subscribeScrips", "inputtoken": "nse_cm|11536", "auction": false }
{ "event": "subscribeFullDepth", "inputtoken": "nse_fo|49081", "auction": false }
{ "event": "snapshotScrips", "inputtoken": "nse_cm|11536", "auction": false }
{ "event": "unsubscribeScrips", "inputtoken": "nse_cm|11536", "auction": false }
{ "event": "subscribeIndices", "inputtoken": "nse_cm|26000", "auction": false }
```

4. Receiving & De-framing Binary Frames

One binary frame can hold **several packets** laid end to end (a "batch"). Each packet starts with its own length, so you split the frame by walking it:

```
frame: [ packet 1 ][ packet 2 ][ packet 3 ] ...  
|  
+-- first 2 bytes = message_length (uint16 LE) = size of THIS packet,  
header included. This is also the packet's own header field.
```

Read the first 2 bytes to get the packet size, slice out that many bytes, advance, repeat until the frame ends:

```
offset = 0  
while offset + 2 <= frame.length:  
    size = read_u16_le(frame, offset) // length of this packet (incl. header)  
    if offset + size > frame.length:  
        break // truncated tail – stop  
    handle(frame[offset : offset + size])  
    offset += size
```

Routing each packet. Read `message_code` (`uint16` LE at offset 2) and `level` (`uint8` at offset 5) from the header, then dispatch:

Condition (checked in order)	Packet type	Section
<code>message_code == 7207</code>	Index	§7.1
<code>message_code == 6511</code>	Market OPEN (header only)	§7.4
<code>message_code == 6521</code>	Market CLOSE (header only)	§7.4
<code>level == 1</code>	Mini Touch Line	§7.3
<code>level ∈ {2, 4, 8, 16}</code>	Market Picture (full)	§7.2
otherwise	ignore	—

Important: market-data packets are routed by the `level` byte, not by `message_code`. (Market-picture packets carry `message_code == 7208`, but you should branch on `level`.) For `level == 4` (touch line), treat the depth as exactly **1 bid + 1 ask** row regardless of the counts in the body.

5. Enums & Constants

`// Exchange – transmitted as a signed byte in the header (exchange_id).`

```
enum Exchange /* : int8 */ {
    NONE = 0, // "none"
    NSE_CM = 1, // "nse_cm"
    NSE_FO = 2, // "nse_fo"
    CDE_FO = 3, // "cde_fo"
    NSE_COM = 4, // "nse_com"
    BSE_CM = 5, // "bse_cm"
    BSE_FO = 6, // "bse_fo"
    BSE_CD = 7, // "bse_cd"
    BSE_CO = 8, // "bse_co"
    MCX_FO = 9, // "mcx_fo"
    NCD_CO = 10, // "ncd_co"
};
```

`// Level (depth) – transmitted as a u8 in the header (level).`

```
enum Level /* : uint8 */ {
    MINI_TOUCH_LINE = 1,
    TOUCH_LINE = 4,
    DEPTH = 8,
    FULL_DEPTH = 16,
};
```

Message codes

Code	Meaning	Frame type
1117	Auth response	JSON text
6511	Market open	binary (header only)
6521	Market close	binary (header only)
7207	Index data	binary
7208	Market picture	binary (route by level)

Dividers (price scale). Always prefer the divider delivered in the auth response

(`exchanges[*].divider`). Typical values:

Exchange	Divider
NSE_CM, NSE_FO, BSE_CM, BSE_FO, MCX_FO	100
NSE_COM, BSE_CO	10000
CDE_FO, BSE_CD	10000000
NCD_CO	1

If an exchange is missing from the auth response, default its divider to 100 .

6. Field Conventions

- **Endianness / packing:** all integers are **little-endian**; all structs are **packed**

with no padding (`#pragma pack(1)`).

- **Prices** are scaled integers → **display value = raw / divider** (per exchange, from the auth response).

- **net_chg_percent** is scaled by **100** → **display = raw / 100** (independent of the divider).

- **net_chg** is scaled by the **divider** → **display = raw / divider**.

- **precision** (when present) is the number of decimal places to format the value to.

- **Strings** are fixed-size byte arrays, UTF-8, **NUL-padded**. Decode up to the first `\0` (or array end) and trim surrounding whitespace.

- **Timestamps** are integer epoch values exactly as sent by the exchange; treat them as

opaque 64-bit integers unless you know the exchange's epoch/units.

- Fields named with a leading underscore (e.g. `_market_cap`) are **present on the wire**

but **not used** by the reference client — they are documented so your offsets line up.

7. Wire Structures

All structures below are little-endian and packed.

```
#pragma pack(push, 1)
```

Common header (every binary packet)

```
typedef struct {  
    uint16_t message_length; // total packet size incl. header (also the de-frame length)  
    uint16_t message_code; // 7207 index, 7208 market picture, 6511/6521 market open/close  
    int8_t exchange_id; // enum Exchange  
    uint8_t level; // enum Level  
    uint8_t auction_flag; // 0 = normal, > 0 = auction  
    uint8_t seq_no; // sequence number (significant in delta mode; informational here)  
    uint8_t bitmask_length; // delta bitmask length (0 for native_batch full messages)  
} MsgHeader; // 9 bytes
```

7.1 Index message — `message_code == 7207`

```

typedef struct {
    MsgHeader hdr; // @0 (9 bytes)
    uint32_t token; // @9 instrument token
    int32_t open_price; // @13 / divider
    int32_t close_price; // @17 / divider (previous close / closing index)
    int32_t high_price; // @21 / divider
    int32_t low_price; // @25 / divider
    int32_t index_value; // @29 / divider (current value / LTP)
    uint64_t last_trade_time; // @33 epoch
    int32_t yearly_high; // @41 / divider
    int32_t yearly_low; // @45 / divider
    int32_t net_chg_percent; // @49 / 100
    double _market_cap; // f64 (present, unused by client)
    uint8_t precision; // decimal places for display
    int32_t multiplier; // / divider (Unused)
    char name[21]; // index name, UTF-8, NUL-padded
} IndexMessage;

```

Derived value: $\text{change} = \text{index_value} - \text{close_price}$.

7.2 Market Picture (full) — $\text{level} \in \{2, 4, 8, 16\}$

A **144-byte fixed body**, followed by $\text{buy_depth_count} + \text{sell_depth_count}$ depth rows.

The first buy_depth_count rows are **bids (buy)**; the remaining sell_depth_count rows are **asks (sell)**.

For $\text{level} == \text{TOUCH_LINE}$ (4), force the counts to **1 bid + 1 ask** (read exactly one of each) regardless of the values in $\text{buy_depth_count} / \text{sell_depth_count}$.

```
typedef struct { // offset within row
    int64_t qty; // @0
    int32_t price; // @8 / divider
    int32_t number_of_orders; // @12
} DepthRow; // 16 bytes

typedef struct {
    MsgHeader hdr; // @0 (9 bytes)
    uint32_t token; // @9
    int64_t total_buy_qty; // @13
    int64_t total_sell_qty; // @21
    int64_t volume_traded_today; // @29
    int64_t last_trade_time; // @37 epoch
    int64_t last_update_time; // @45 epoch
    uint32_t open_price; // @53 / divider
    uint32_t close_price; // @57 / divider
    uint32_t high_price; // @61 / divider
    uint32_t low_price; // @65 / divider
    uint32_t last_traded_price; // @69 / divider
    int64_t last_trade_qty; // @73
    uint32_t avg_trade_price; // @81 / divider
    uint32_t _indicative_close; // @85 (present, unused)
    uint32_t buy_depth_count; // @89 number of bid rows that follow
    uint32_t sell_depth_count; // @93 number of ask rows that follow
    int16_t _trading_status; // @97 (present, unused)
    int32_t net_chg_percent; // @99 / 100
    uint32_t open_interest; // @103
    double total_traded_value; // @107 f64 / divider
    int32_t net_chg; // @115 / divider
    uint32_t upper_circuit; // @119 / divider
    uint32_t lower_circuit; // @123 / divider
    uint32_t yearly_high; // @127 / divider
```

```

uint32_t yearly_low; // @131 / divider
uint32_t market_lot; // @135
uint8_t precision; // @139
uint32_t multiplier; // @140
// DepthRow depth[buy_depth_count + sell_depth_count]; // @144 ...
} MarketPicture; // 144-byte fixed body + depth rows

```

Reading the depth rows: start at offset 144 and read

buy_depth_count + sell_depth_count DepthRows, but stop early if you reach
 hdr.message_length (the packet may carry fewer rows than the counts imply).

7.3 Mini Touch Line — level == 1

A compact **54-byte** message with no depth rows.

```

typedef struct {
  MsgHeader hdr; // @0 (9 bytes)
  uint32_t token; // @9
  int64_t last_trade_time; // @13 epoch
  uint32_t last_traded_price; // @21 / divider
  int64_t last_trade_qty; // @25
  uint32_t close_price; // @33 NOTE: NOT divided by the divider (see below)
  int32_t net_chg_percent; // @37 / 100
  int32_t net_chg; // @41 / divider
  uint32_t market_lot; // @45
  uint8_t precision; // @49
  uint32_t multiplier; // @50
} MiniTouchLine; // 54 bytes

```

Quirk: in the reference client, close_price in this message is used ****as-is** and is
 NOT scaled by the divider**, unlike every other price. Reproduce this behavior to match
 the reference, or apply the divider if you determine the raw value is scaled.

7.4 Market status (header only)

Market open/close notifications carry **only the 9-byte header** (MsgHeader):

- `message_code == 6511` → market **OPEN** for `exchange_id`.
- `message_code == 6521` → market **CLOSE** for `exchange_id`.

```
typedef struct {
```

```
MsgHeader hdr; // message_code = 6511 (open) | 6521 (close); exchange_id identifies the segment
```

```
} MarketStatus; // 9 bytes
```

```
#pragma pack(pop)
```

8. Consuming — A Normalized Model

It is convenient to decode every message type into one normalized record for your UI. The reference client maps all messages into the following shape (names shown for orientation; adapt to your language):

```

token // instrument token

exchange // enum Exchange

symbol, name // identifiers (name is the index/instrument name)

ltp // last traded price (index_value for indices)

ltt // last trade time (epoch)

lut // last update time (epoch)

ltq // last trade qty

atp // average trade price

open, high, low, close

net_chg, net_chg_percent

oi // open interest

volume_traded_today

total_buy_qty, total_sell_qty

buy[], sell[] // arrays of { qty, price, no_of_orders }

isIndex // true for index messages

auctionStatus // from header auction_flag

upperCircuit, lowerCircuit

yearlyHigh, yearlyLow

traded_value // total traded value

```

Suggested map key (so updates replace prior state for the same instrument):

Case	Key
Index (isIndex)	exchange name
Auction (auctionStatus > 0)	exchange token auction
Otherwise	exchange token

9. End-to-End Flow

1. **Connect.** Build `wss://<host>/mfeed` and open the WebSocket; set binary type to

`arraybuffer` .

2. **Authenticate.** On open, send the auth JSON ([§2.2](#)) with

```
"format": "native_batch" .
```

3. **Receive auth response.** On the text frame with `message_code == 1117` , store every `exchanges[*].divider` .

4. **Subscribe.** e.g. send

```
{ "event": "subscribeScrips", "inputtoken": "nse_cm|11536", "auction": false } .
```

5. **Receive a binary frame.** Treat it as a batch.

6. **De-frame** it into packets ([§4](#)).

7. **Route** each packet by `message_code / level` .

8. **Parse** the matching struct ([§7](#)).

9. **Scale** prices by the exchange divider; format with `precision` .

10. **Update** your normalized model ([§8](#)) and render.

11. On **close**, reconnect and repeat from step 2 (re-auth + re-subscribe).

10. Quick Reference

Item	Value
Transport	WebSocket, single connection
Control plane	JSON text frames
Data plane	Binary frames (<code>arraybuffer</code>), little-endian, batched
Endianness	Little-endian everywhere
Struct packing	Packed, no padding (<code>#pragma pack(1)</code>)
Header size	9 bytes
Default path	<code>/mfeed</code>
Auth response code	1117
Index code	7207
Market picture code	7208 (route by <code>level</code>)
Market open / close	6511 / 6521
Index body	<code>IndexMessage</code>
Market picture fixed body	144 bytes + <code>DepthRow</code> × (buy+sell)
Mini touch line	54 bytes, no depth
<code>DepthRow</code>	16 bytes (<code>qty i64</code> , <code>price i32</code> , <code>orders i32</code>)
Price decode	<code>raw / divider</code> (divider from auth)
<code>net_chg_percent</code> decode	<code>raw / 100</code>